Rockchip RT-Thread SPIFlash Developer Guide

ID: RK-KF-YF-080

Release Version: V1.6.0

Release Date: 2025-08-11

Security Level: ☐Top-Secret ☐Secret ☐Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily describes the principles and usage methods of ROCKCHIP RT-Thread SPI Flash.

Product Version

Chipset	Kernel Version
All chip products using RK RT-Thread SDK	RT-Thread

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version Number	Author	Dat	Change Description
V1.0.0	Lin Dingqiang	2020-02-21	Initial version
V1.0.1	Chen Mouchun	2020-03-05	Modified unordered list indentation
V1.1.0	Lin Dingqiang	2024-01-18	Added chip FSPI/SFC key features, software framework introduction
V1.2.0	Lin Dingqiang	2024-03-26	Added RK2118 support, SPI Nand support instructions
V1.3.0	Lin Dingqiang	2024-06-03	Optimized SPI Nand solution
V1.4.0	Lin Dingqiang	2024-07-09	Added SPI Nand read-only block device support
V1.5.0	Lin Dingqiang	2025-07-12	Added RK1820, RK2116 support
V1.6.0	Lin Dingqiang	2025-08-11	Added SPI Nor Precautions

Contents

Rockchip RT-Thread SPIFlash Developer Guide

1. Introduction
    - 1.1 Supported Devices
    - 1.2 Main Controller
    - 1.3 XIP Technology for NOR Flash
    - 1.4 SPI Flash Driver Framework
    - 1.5 Key Features of Chip FSPI Module
2. SPI NOR Flash
    - 2.1 Precautions
    - 2.2 Configuration
        - 2.2.1 Framework Overview
        - 2.2.2 Driver Configuration
        - 2.2.3 Partition Table Configuration
        - 2.2.4 Partition Information Generation, Parsing, and Registration of Partition Devices
        - 2.2.5 elm-fat File System and Partition Mounting File System
    - 2.3 XIP Implementation Guidelines
        - 2.3.1 Adding XIP Support
        - 2.3.2 XIP Usage Process Control
    - 2.4 Function Interface Invocation Example
        - 2.4.1 SNOR MTD Byte Device
        - 2.4.2 Partition MTD Byte Device
        - 2.4.3 Partition Block Device
    - 2.5 Test Driven Development
        - 2.5.1 Driver Configuration
        - 2.5.2 Test Commands
        - 2.5.3 Test Command Examples
            - 2.5.3.1 Read Rate
            - 2.5.3.2 Quad Flash Signal Testing
            - 2.5.3.3 Octal Flash Signal Testing
    - 2.6 Common Issues
3. SPI NAND
    - 3.1 Important Notice
        - 3.1.1 Introduction to RK SPI Nand Storage Solution
    - 3.2 Configuration
        - 3.2.1 Framework Overview
        - 3.2.2 Driver Configuration
        - 3.2.3 FTL Algorithm
            - 3.2.3.1 DHARA FTL Algorithm
            - 3.2.3.2 Mini FTL Driver
        - 3.2.4 Partition Table Configuration
            - 3.2.4.1 Partition Planning
            - 3.2.4.2 RK Partition Expansion
            - 3.2.4.3 GPT Partition Expansion
            - 3.2.4.4 Customizing ROOT Partition Expansion
        - 3.2.5 Partition Information Parsing and Registering Partition Devices
    - 3.3 PC Tool Burning
    - 3.4 Function Interface Invocation Example
        - 3.4.1 MTD Byte Device - Basic Interface
        - 3.4.2 Mini FTL Interface - OTA Interface/File System
        - 3.4.3 DHARA FTL Interface - File System Interface
    - 3.5 File System
        - 3.5.1 elm FatFs Support
            - 3.5.1.1 Partition Mounting

3.5.1.2 Online Low-Level Formatting of Partitions
3.5.1.3 Offline Pre-creation of elm-fat Image
3.5.1.4 Offline Pre-fabrication of DHARA Images
3.6 Test-Driven Development
3.6.1 Test Driven Infrastructure
3.6.2 flash_stress_test Read/Write Stress Test
3.6.3 Power Lost Test: Unexpected Power Loss
3.7 Frequently Asked Questions
3.7.1 Enabling SPI Nand on RK2118 Results in System Hang
3.7.2 SPI Nand Does Not Support RK Firmware.img Packaged Image Upgrade Solution
3.7.3 SPI Nand Power Loss Issues
3.7.4 SPI Nand Power Failure Reference Code

1. Introduction

1.1 Supported Devices

Supported SPI Nor:

SPI Nor possesses the advantages of small packaging, faster read speed compared to other small-capacity non-volatile storage, fewer pins, and a simple protocol. Commonly available SPI Nor supports 1-line, 2-line, and 4-line transfers, and it has characteristics such as non-volatile flip, byte addressing, and no waiting delay in read operations (data is transmitted on the next beat after sending cmd and address). Therefore, it supports the XIP technology (Excute In Place).

The RK RTOS SPI flash framework provides a universal SPI Nor interface and an automated XIP solution.

Supported SPI Nand:
SPI Nand and SPI Nor use the same controller on the RK platform, and RK SDK also provides targeted support for RT-Thread. However, the use of SPI Nand depends on the FTL algorithm or a file system that supports the FTL algorithm. The FTL algorithm consumes a large amount of ram/code size, has average stability, and SPI Nand has high requirements for power supply stability when the power is off. It is not friendly to MCU products without batteries and has a higher risk of abnormal power-off, so it is not recommended to use SPI Nand storage. It is only recommended to be used as a read-only firmware extension storage in a few scenarios.

1.2 Main Controller

The RK platform SPI Flash can choose from controllers including FSPI, SFC, and SPI, with three schemes.

FSPI (Flexible Serial Peripheral Interface) is a flexible serial transmission controller with the following main features:

- Supports SPI Nor, SPI Nand, SPI protocol Psram, and SRAM
- Supports Standard SPI (single line), Dual SPI, Quad SPI, with some versions supporting Octal SPI
- Supports SDR (single-edge transmission), with some versions supporting DTR (dual-edge transmission)
- XIP technology
- DMA transfer (built-in DMA)

SFC (Serial Flash Controller) is a serial transmission controller with the following main features:

- Supports SPI Nor, SPI Nand, SPI protocol Psram, and SRAM
- Supports Standard SPI (single line), Dual SPI, Quad SPI
- Supports SDR (single-edge transmission)
- DMA transfer (built-in DMA)

SPI (Serial Peripheral Interface) is a general-purpose serial transmission controller with the following main features:

- Supports SPI Nor, SPI Nand, SPI protocol Psram
- Supports Standard SPI (single line)
- Supports SDR (single-edge transmission)
- DMA transfer (external DMA)

1.3 XIP Technology for NOR Flash

XIP (eXecute In Place) refers to the execution of instructions directly from the memory space mapped to the CPU, meaning that applications can run directly within the flash memory without the need to read the code into the system RAM. Consequently, the execution address for the code running in place is the corresponding mapped address. Since SPI NOR XIP only supports reading, the code segment and read-only information can only be placed in the SPI NOR.

FSPI not only supports CPU XIP access to SPI flash but also allows other modules such as DSP to acquire flash data in a similar manner, akin to accessing a "read-only SRAM" space. For detailed information on FSPI, refer to the FSPI section in the TRM.

Implementation of SPI NOR read function interfaces under the XIP scheme:

- Support for:
    - XIP Read, accessing XIP-mapped space to obtain data without redundant configuration register actions and supports prefetching, resulting in high efficiency and overall faster speed
    - CPU Read, configuring controller registers to initiate transmission, with the CPU moving FIFO data
- The default read interface is implemented as "XIP Read"
- Supports configuration of the starting range for "XIP Read" implementation, RT_SNOR_XIP_DATA_BEGIN, for example, defined as 327680, which means:
    - The space [0: 327680) is implemented with "CPU Read"
    - The space [327680, end] is implemented with "XIP Read"

1.4 SPI Flash Driver Framework

Considering the need to adapt to FSPI, SFC, and SPI controllers, an abstraction layer for the controllers is created, dividing the entire driver framework into four levels, using SPI Nor as an example:

- MTD Framework Layer
- RTOS Driver Layer, completing the following logic:
    - RTOS device framework registration
    - Registering the controller and operation interfaces to the HAL_SNOR protocol layer
    - Encapsulating read-write-erase interfaces for users
- HAL_SNOR protocol layer based on SPI Nor transmission protocol
- Controller Layer

RT-Thread Implementation Based on FSPI Controller:

- OS Driver Layer: drv_snor.c implements:
  - Encapsulates SPI_Xfer based on FSPI HAL layer read-write interfaces, and registers FSPI host and SPI_Xfer to the HAL_SNOR protocol layer
  - Encapsulates read-write-erase interfaces provided by the HAL_SNOR protocol layer
  - Registers OS device driver to the MTD framework layer
- Protocol Layer: hal_snor.c in the HAL development package implements the protocol layer for SPI Nor flash
- Controller Layer: hal_fspi.c in the HAL development package implements the FSPI controller driver code

RT-Thread Implementation Based on SPI Controller:

- OS Driver Layer: drv_snor.c implements:
  - Encapsulates SPI_Xfer based on SPI OS driver read-write interfaces, and registers SPI host and SPI_Xfer to the HAL_SNOR protocol layer;
  - Encapsulates read-write-erase interfaces provided by the HAL_SNOR protocol layer
  - Registers OS device driver to the MTD framework layer
- Protocol Layer: hal_snor.c in the HAL development package implements the protocol layer for SPI Nor flash
- Controller Layer: hal_spi.c in the HAL development package implements the SPI controller low layer driver code, and SpiDevice.c implements the RTOS SPI DRIVER device registration and interface encapsulation

Notes:

1. The above implementations correspond one-to-one with the attached diagrams, which can be read in conjunction
2. Since the RK SPI DMA transfer-related code is in the OS Driver layer, and the SPI controller is not only used for SPI Nor but also supports many other devices, there is hardware resource boundary protection. Therefore, in the SPI Flash framework, the SPI controller should not directly interface with the HAL layer hal_spi.c driver, but should use the SPI interfaces in the OS Driver.

1.5 Key Features of Chip FSPI Module

Only statistics for SOC chips using open-source storage are included:

SOC	FSPI/SFC IO max rate	FSPI/SFC data io lines	Support XIP
RK2108/Pisces	150MHz	x1\2\4 SDR	Y
RK2206	150MHz	x1\2\4 SDR	Y
RK2118	SDR: 133MHz DTR: 80MHz	x1\2\4 SDR x4 DDR with/without DQS x8 DDR with/without DQS	Y
RK1820	SDR: 133MHz DTR: 75MHz	x1\2\4 SDR x4 DDR with/without DQS	N

SOC	FSPI/SFC IO max rate	FSPI/SFC data io lines	Support XIP
RK2116	SDR: 125MHz	x1\2\4 SDR	Y

Explanation:

- For AP chip support status, refer to the document **"Rockchip_Developer_Guide_Linux_Flash_Open_Source_Solution_EN.pdf"**
- Commonly referred to as x1 as Standard SPI, x2 as Dual SPI, x4 as Quad SPI, x8 as Octal SPI
- Quad SPI chips are configured by default in SDR mode, Octal SPI chips are configured by default in DDR with DQS

2. SPI NOR Flash

2.1 Precautions

- The rate configuration should comply with the constraints of the SPI Nor manual, especially the AC Electrical Characteristics FR read rate indicators. Note:
  - Some SPI Nor devices, such as Macronix, XMC, ISSI, have different FR indicators for different read commands. The constraints should match the used command, which is reflected in the system startup log `readCmd`.

2.2 Configuration

2.2.1 Framework Overview

Abbreviation	Main Supported Particle Types	Controller Driver	Flash Framework
SPI Nor	SPI Nor	hal: lib/hal/src/hal_fspi.c RT-Thread: bsp/rockchip/commob/drv_snor.c	hal: lib/hal/src/hal_snor.c

Notes:

- The root directory of RT-Thread HAL source code is located at `bsp/rockchip/common/hal`.
- The supported SPI Nor particles can be found in the source code `lib/hal/src/hal_snor.c`, with other particle support in the same directory level.
- Particles can be matched via flash ID or flash silkscreen.
- The software identifies specific particles through flash ID, with the default configuration set to the highest supported bus width for the controller. If a lower bus width is required, the driver macro switch must be configured.

2.2.2 Driver Configuration

Before proceeding with this configuration, it is necessary to identify the type of SPI Flash controller used on the hardware to select the appropriate scheme.

Basic Configuration

Menuconfig configuration entry:

```
RT-Thread rockchip common drivers  -->
    [*] Enable ROCKCHIP SPI NOR Flash
    (80000000) Reset the speed of SPI Nor flash in Hz
    [ ]   Set SPI Host DUAL IO Lines /* If the FSPI controller only reserves
IO0~1, Dual mode should be used */
            Choose SPI Nor Flash Adapter (Attach FSPI controller to SNOR) -->
```

Configuration Description:

```
RT_USING_MTD_NOR=y
RT_USING_SNOR=y
RT_SNOR_SPEED=80000000 /* IO interface speed */
# RT_SNOR_DUAL_IO=n /* Default not configured, Quad SPI limited to Duad SPI usage
*/
# RT_SNOR_XIP_DATA_BEGIN=0 /* Default not configured, XIP read interface
implementation start address, refer to "Nor Flash XIP Technology" for details */
RT_USING_SNOR_FSPI_HOST=y /* FSPI controller scheme */
# RT_USING_SNOR_SFC_HOST=y /* SFC controller scheme */
# RT_USING_SNOR_SPI_HOST=y /* SPI controller scheme */
# RT_SNOR_SPI_DEVICE_NAME="spi2_0" /* SPI controller scheme, specify the target
controller */
RT_USING_DFS_MNTTABLE=y               # Enable DFS mnt table automatic mount
support
RT_USING_DFS_ELMFAT=y                 # Enable elm FatFs
RT_DFS_ELM_MAX_SECTOR_SIZE=4096       # Set elm FatFs sector size, fixed at 4KB
```

2.2.3 Partition Table Configuration

Refer to the "Partition Table Configuration" section in the "Rockchip_Developer_Guide_RT-Thread_EN.pdf" located in the rockchip directory.

Notes:

- SPI Nor defaults to only register partitions with FLAG [8, 9] where the property is 1, 2, or 3 as RT-Thread devices. For example, the root partition typically has Flag=0x305, and after the flash driver runs, it is registered as the root block device.

2.2.4 Partition Information Generation, Parsing, and Registration of Partition Devices

The firmware packaged by RK RT-Thread SDK will generate an RK_PARTITION table at the front 2KB position of the flash. The partition information can be adjusted by modifying the corresponding setting.ini.

The partition table downloaded with the firmware to the SPI Nor will be detected, parsed, and registered after the SPI Nor is initialized (implemented in the snor_partition_init function in drv_snor.c).

You can check whether the corresponding partitions are registered successfully with the following command:

```
msh />list_device
device          type         ref count
------- -------------------- ----------
root     Block Device         1           /* Partition name root, partition type
block device, Nor flash supports setting.ini to modify and set as MTD device */
snor     MTD Device           0           /* SPI Nor root storage device,
partition read and write operations ultimately access this node to complete read,
write, and erase operations */
```

2.2.5 elm-fat File System and Partition Mounting File System

RT-Thread elm-fat File System Support

```
RT-Thread Components --->
    Device virtual file system  --->
    [*] Using device virtual file system
    [*]   Using mount table for file system /* Implement corresponding
registration partition table to enable automatic mounting of partitions at power-
up */
    [*]   Enable elm-chan fatfs /* FAT file system */
        elm-chan's FatFs, Generic FAT Filesystem Module  --->
        (4096) Maximum sector size to be handled. /* For SPI Nor products, it
must be modified to 4096 */
```

If you enable the partition automatic mounting file system macro `CONFIG_RT_USING_DFS_MNTTABLE`, you can add corresponding partition registration information in mnt.c, for example, the "root" partition to the "/" directory:

```
const struct dfs_mount_tbl mount_table[] =
{
    {"root", "/", "elm", 0, 0},
    {0}
};
```

If you wish to design your own file system mounting process, you can also achieve file system mounting through the following code:

```
dfs_mount("root", "/", "elm", 0, 0)
```

Partition mounting command:

```
mount root / elm
```

elm-fat Image Creation:

It is recommended to refer to the `./bsp/rockchip/tools/mkroot.sh` script for implementation, with the key command `mkfs.fat`

2.3 XIP Implementation Guidelines

Previously, it has been introduced that SPI Nor supports XIP functionality. If an SPI Nor solution implemented with an FSPI master is chosen, the XIP feature will be automatically enabled. Below are some guidelines to be aware of when applying XIP in product applications.

2.3.1 Adding XIP Support

When using the SPI Flash scheme implemented with FSPI, and configuring it according to the "Driver Configuration" section regarding the FSPI configuration method, the SPI Flash will default to using the XIP function. To disable this feature, one should disable the configuration "Enable FSPI XIP".

2.3.2 XIP Usage Process Control

The RK SPI Flash framework will automatically switch the XIP feature on and off in required scenarios, and customers do not need to call the control interface. Details are as follows:

XIP Off

Due to the long duration of SPI Nor erase/write operations, SPI Nor does not support erase/write under XIP. Therefore, when there are erase and write requests for SPI Nor flash, such as file system write requests, the software will call the XIP suspend interface to switch the SPI nor controller FSPI to normal mode. During this period, XIP functionality will not be available. The complete suspend XIP switching process is as follows:

1. Notify all master modules affected by XIP disable to suspend XIP operations
2. Disable global interrupts to prevent interrupts from causing the CPU to execute XIP code stored in SPI Nor
3. Disable XIP

XIP On

After the SPI Nor flash erase/write is completed and FSPI is idle, the SPI Flash driver will resume XIP functionality. That is, FSPI will keep XIP functionality enabled when there are no erase/write operations. The complete XIP resume process is as follows:

1. Enable XIP
2. Enable global interrupts

3. Notify all XIP suspend devices to resume using XIP

The operation entries mentioned above are as follows:

```
static rt_base_t snor_xip_suspend(void)
static void snor_xip_resume(rt_base_t level)
```

Summary

- If the SPI Nor chip is driven by the FSPI controller, the XIP feature is enabled by default
- The SPI Nor controller FSPI defaults to enabling XIP functionality when idle, allowing devices supporting XIP pathways to access XIP memory-mapped addresses to obtain data from SPI Nor
- The SPI Nor controller FSPI disables XIP functionality when performing erase and write operations
- When the SPI Nor controller FSPI switches XIP on and off, it is necessary to notify the corresponding devices to stop/resume XIP access, as described in the "XIP Off" section above
- When XIP is turned off, global interrupts are also disabled

2.4 Function Interface Invocation Example

2.4.1 SNOR MTD Byte Device

Refer to the _at_snor_test test code in the snor_test.c source code, which mainly involves:

```
snor_device = (struct rt_mtd_nor_device *)rt_device_find("snor");
rt_mtd_nor_erase_block(snor_device, test_lba * block_size, block_size);
rt_mtd_nor_write(snor_device, test_lba * block_size, (const rt_uint8_t
*)pwrite32, block_size);
rt_mtd_nor_read(snor_device, test_lba * block_size, (rt_uint8_t *)pread32,
block_size);
```

Explanation:

- snor is a byte device
- rt_mtd_nor_erase_block: The unit is Bytes, with a minimum alignment of 4096Bytes, which is the Flash sector size
- rt_mtd_nor_write/read: The unit is Bytes, with a minimum alignment of 1 Byte
- RK partition table unit is sector (512B), please pay attention to unit conversion when reading and writing partition table information

2.4.2 Partition MTD Byte Device

As described in the section "Partition Information Generation, Parsing, and Registration of Partition Devices," the SPI Nor driver parses the partition table and registers specific partitions as partition devices. If configured as an MTD partition through setting.ini, it is registered as an MTD byte device by default, with the same interface as the SNOR MTD byte device, primarily for:

```
snor_device = (struct rt_mtd_nor_device *)rt_device_find("xxxx");
rt_mtd_nor_erase_block(snor_device, test_lba * block_size, block_size);
rt_mtd_nor_write(snor_device, test_lba * block_size, (const rt_uint8_t
*)pwrite32, block_size);
rt_mtd_nor_read(snor_device, test_lba * block_size, (rt_uint8_t *)pread32,
block_size);
```

2.4.3 Partition Block Device

As described in the "Partition Information Generation, Parsing, and Registration of Partition Devices" section, the SPI Nor driver parses the partition table and registers specific partitions as partition devices. If configured as a Block partition through setting.ini, it is registered as a block device by default, with the interface referring to the standard read/write method of RT-Thread, with the sector unit being 4KB.

2.5 Test Driven Development

It is recommended to introduce the following testing procedures in the SPI Flash development process to perform simple read and write tests for judgment.

2.5.1 Driver Configuration

```
    RT-Thread bsp test case  --->
        RT-Thread Common Test case  --->
            [*] Enable BSP Common TEST
            [*]   Enable BSP Common SNOR TEST (NEW)
```

If the configuration is successful, there will be a `snor_test` command option in msh.

2.5.2 Test Commands

Entering the command `snor_test` provides detailed instructions. The units for the following commands are all in bytes.

```
1. snor_test write offset size
2. snor_test read offset size loop
3. snor_test erase offset
4. snor_test stress offset size loop
5. snor_test io_stress offset size loop width rw
6. snor_test pm_test
like:
        snor_test write 2097152 4096
        snor_test read 2097152 4096 2000
        snor_test erase 2097152
        snor_test stress 2097152 2097152 5000
        snor_test io_stress 2097152 2097152 5000 4
        snor_test pm_test
```

Explanation:

- offset: The offset address within the flash, not the XIP address space, supports only decimal input, does not support hexadecimal, unit is bytes.
- stress: The data is similar to an incrementing sequence.
- io_stress
  - width: The data type supports various IO status sequences, where width 4 is a sequence dedicated to Quad Flash, and width 8 is a sequence dedicated to Octal Flash.
  - rw: 1-Write only (including erase), 2-Read only, 3-Read and Write.

2.5.3 Test Command Examples

2.5.3.1 Read Rate

```
msh >snor_test read 2097152 4096 2000
[SNOR TEST] read:  30219d00 + 0x0:0x00000200,0x0000000f,0x000000ff,0x00000000,
[SNOR TEST] read:  30219d00 + 0x10:0x000000ff,0x000000cc,0x000000c3,0x000000cc,
[SNOR TEST] read:  30219d00 + 0x20:0x000000c3,0x0000003c,0x000000cc,0x000000ff,
[SNOR TEST] read:  30219d00 + 0x30:0x000000fe,0x000000ff,0x000000fe,0x000000ef,
snor read speed: 143719 KB/s
```

2.5.3.2 Quad Flash Signal Testing

Signal Writing Test:

```
snor_test io_stress 2097152 2097152 5000 4 1
```

Signal Reading Test:

```
snor_test io_stress 2097152 2097152 5000 4 2
```

2.5.3.3 Octal Flash Signal Testing

Signal Writing Test:

```
snor_test io_stress 2097152 2097152 5000 8 1
```

Signal Reading Test:

```
snor_test io_stress 2097152 2097152 5000 8 2
```

2.6 Common Issues

- **How to determine if SPI flash has been successfully mounted?**

  Check for the presence of a snor device node using list_device.

- **What does the error message "Init adapte error ret= -19" indicate?**

  The message "Init adapte error ret= -19" is the result of the storage driver's initialization function snor_adapt. The corresponding error codes are explained as follows:

  a. -1: SPI Nor ID is 0xff, which may indicate that the SPI Nor is not soldered properly. Please check the circuitry and signals.

  b. -19: The chip is not in the support list. Please contact RK engineers to add support for the corresponding chip.

  c. -22: SPI controller not found.

- **Is there any additional power consumption when XIP functionality is always enabled?**

  The FSPI has a time out mechanism. After the main controller is idle for a certain period, it automatically releases cs, and the SPI Nor enters a low-power idle state, so there should not be a significant difference in power consumption.

- **If multiple masters access SPI Nor simultaneously through XIP, will there be any conflicts?**

  When multiple masters access XIP simultaneously, the bus will arbitrate, and the transmission will be completed serially with requests hanging on the bus.

- **Should XIP functionality be used for products that frequently read and write file systems?**

  For products that frequently write to the file system, it is recommended to place the code segment in sram or psram and then disable the FSPI XIP functionality. Otherwise, it will affect the real-time performance of the RTOS system. Similarly, for schemes that run code segments using XIP, the erasing and writing of SPI Nor should be minimized.

- **After enabling XIP functionality, should the file system read be turned off XIP?**

  After enabling XIP functionality, the file system read is implemented directly through XIP, so XIP should not be turned off.

- **Can partitions be mounted if there is no partition table on the SPI Nor?**

  Flash partitions must be solidified at the front end of the flash. RK's current SPI Nor flash storage driver only implements partition parsing and mounting solutions that are solidified in the flash.

3. SPI NAND

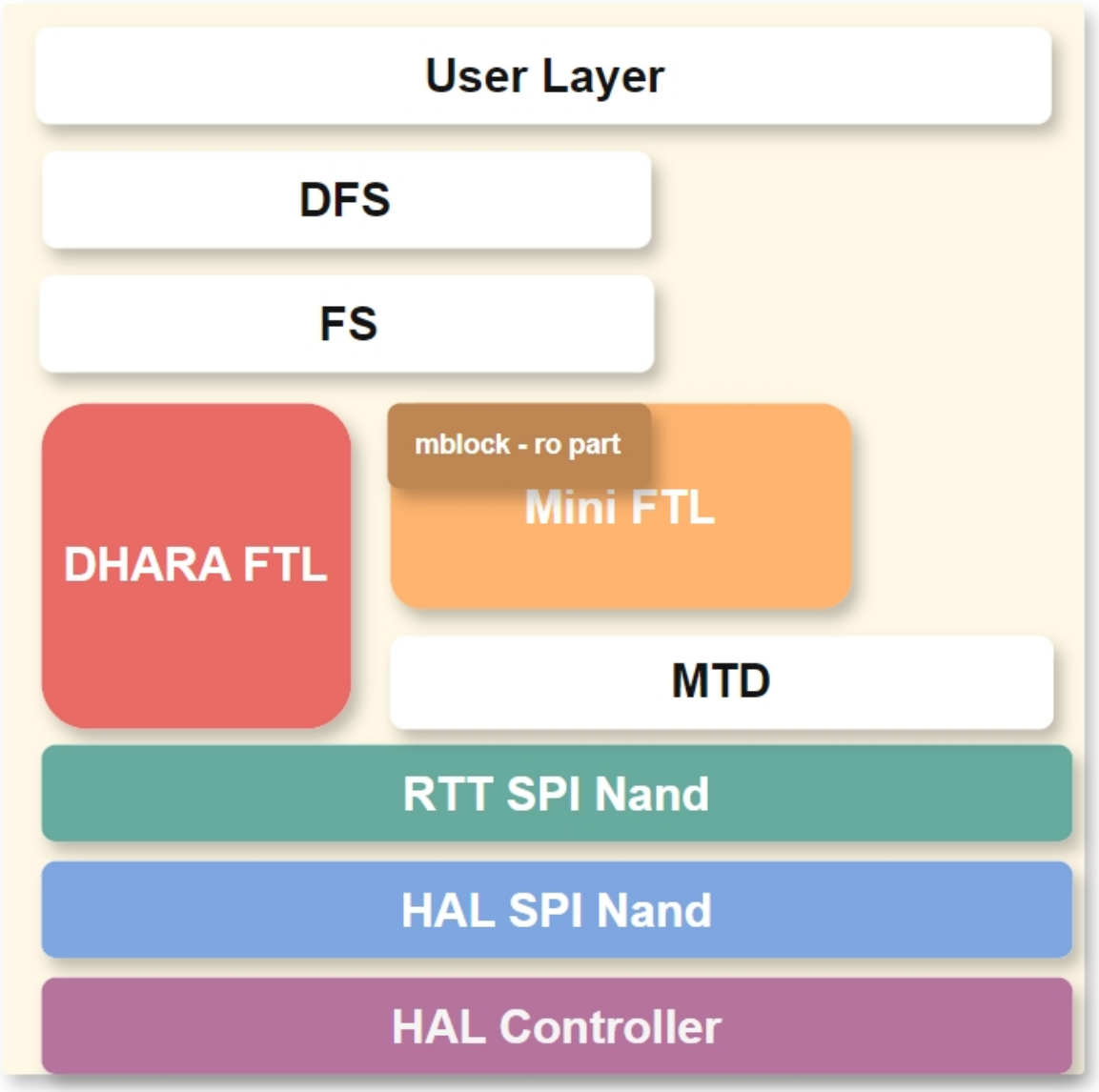
3.1 Important Notice

3.1.1 Introduction to RK SPI Nand Storage Solution

Supported File Systems	Supported Programming Methods
FATFS	USB Upgrade, Programmer Upgrade, OTA Upgrade

3.2 Configuration

3.2.1 Framework Overview



Abbreviation	Main Supported Particle Types	Driver
SPI Nand	SPI Nand	DHARA FTL: bsp/rockchip/common/dhara/ RTT Mini FTL: bsp/rockchip/common/mini_ftl.c RTT SPI Nand: bsp/rockchip/common/drv_spinand.c HAL SPI Nand: lib/hal/src/hal_spinand.c HAL Controller: lib/hal/src/hal_fspi.c

Instructions:

- The root directory of the RT-Thread HAL source code is located at `bsp/rockchip/common/hal`
- The supported SPI Nand particles can be queried from the source code `lib/hal/src/hal_spinand.c`, with additional particle support in the same directory
- Particles can be matched with the source code using flash id or flash silkscreen printing
- The software identifies specific particles through flash id, and the software is configured by default to support the controller's highest transmission line width. If a lower line width is required, the driver macro switch needs to be configured.

3.2.2 Driver Configuration

Before proceeding with this configuration, it is necessary to identify the type of SPI Flash controller used on the hardware to select the appropriate scheme;

Basic Configuration

Menuconfig configuration entry:

```
RT-Thread rockchip common drivers  -->
    [*] Enable ROCKCHIP SPI Nand Flash
    (80000000) Reset the speed of SPI Nand flash in Hz
    (y)   Support mini ftl
          Choose SPI Nor Flash Adapter (Attach FSPI controller to SNOR) -->
            Attach FSPI controller to SPINAND
    [ ]     extend fspi cs1 as SPINAND device (NEW)
```

Configuration Explanation:

```
RT_USING_SNOR=n                         # Disable SPI Nor driver, typically only
one type of SPI Flash is used for peripherals
RT_USING_MTD_NAND=y
RT_USING_SPINAND=y
RT_SPINAND_SPEED=80000000               # IO interface speed
RT_USING_SPINAND_FSPI_HOST=y            # FSPI controller scheme
RT_USING_FTL=y                          # Enable SPI Nand FTL storage management
algorithm
RT_USING_DHARA=y                        # Enable SPI Nand DHARA FTL storage
management algorithm (compatible with Mini FTL)
RT_USING_DFS_MNTTABLE=y                 # Enable DFS mnt table automatic mount
support
RT_USING_DFS_ELMFAT=y                   # Enable elm FatFs
RT_DFS_ELM_MAX_SECTOR_SIZE=2048         # Set elm FatFs sector size, set to flash
pagesize, usually 2KB/4KB, refer to the chip manual
```

Explanation:

- In addition to the above configurations, it should also be confirmed whether XIP support has been disabled, "XIP = 'N'" in the rtconfig.py file.

3.2.3 FTL Algorithm

3.2.3.1 DHARA FTL Algorithm

Dhara (pronounced as: Dæ:ra) is an open-source FTL algorithm designed for managing NAND flash memory in resource-constrained systems. It offers a variable block interface with standard read and write operations. It features the following additional functionalities:

- Wear leveling: The erase count of any two blocks differs by at most 1.
- Garbage Collection (GC) ratio can be adjusted by sacrificing redundant space to optimize GC speed.
- Data integrity: Operations on writing (and trimming) logical sectors are atomic. In case of power loss, the state will roll back to the last synchronization point. Synchronization points occur at fixed time intervals but can also be actively synchronized when needed.
- Real-time performance: All operations (including startup) have a worst-case time complexity of O(log n), where n is the chip size, assuming an even distribution of bad blocks.

After enabling the DHARA FTL algorithm, the partition block device read/write interface of SPI Nand registers as the DHARA read/write interface, with sector size being pagesize, typically 2KB/4KB.

Limitations of the algorithm:

- By default, it only supports the entire Nand space as the FTL algorithm management space.

Current application directions:

- Single FSPI controller, SPI Nand as the primary storage for firmware images and user data.
- Single FSPI controller, CS0 SPI Nor as the primary storage for firmware images, with XIP usage disabled, and CSN1 SPI Nand for storing a large amount of user data.
- Multiple FSPI controllers, FSPI0 SPI Nor as the primary storage for firmware images, supporting FSPI1 with an additional SPI Nand for storing a large amount of user data.

3.2.3.2 Mini FTL Driver

Mini FTL, in fact, is not a standard FTL layer; it provides an interface that only implements the most basic bad block management, primarily used to support firmware upgrades for partitions. It registers a partition named "root" in the partition table as a block device with the same name, for mounting a read-only file system.

3.2.4 Partition Table Configuration

3.2.4.1 Partition Planning

- Partitions must start and be sized to align with 256KB, compatible with 128KB/256KB flash block sizes.
- Spare bad block replacement space must be reserved within partitions, except for the partition table partition, other small partitions should reserve 1-2 flash blocks, and large partitions exceeding 8MB should reserve 1% redundant space.
- It is recommended to reserve a 2MB tail space, compatible with 128KB/256KB flash block bad block table space 4 blocks, and backup partition table space 1 good block.
- It is suggested that the OTA partition planning should be A/B partitioned.

3.2.4.2 RK Partition Expansion

RK's custom partition table, structurally similar to GPT, consumes fewer resources and initializes faster, primarily used in RV1107/8 platforms and MCU platforms.

RK SPI Nand solution defaults to register the last partition as DHARA FTL management partition, which is typically the root user partition used for mounting the file system.

3.2.4.3 GPT Partition Expansion

Since RK SDK supports GPT partition expansion, it also supports GPT partition expansion, and by default, registers the last partition as the DHARA FTL management partition, which is typically the root user partition used for mounting the file system.

3.2.4.4 Customizing ROOT Partition Expansion

When not utilizing partition table partition expansion, it is possible to implement the following ROOT single partition definitions in the drv_spinand.c source code:

```
RT_ROOT_PART_OFFSET
RT_ROOT_PART_SIZE
```

Explanation:

- Units are sectors, with 512B per sector
- Consider referencing the RK3308 RT-Thread build.sh implementation and compilation script binding definition scheme.

3.2.5 Partition Information Parsing and Registering Partition Devices

After successful initialization of SPI Nand, parse the partition table structure and register the last partition.

You can check whether the corresponding partition has been registered successfully with the following command:

```
msh />list_device
device          type         ref count
------- -------------------- ----------
userdata Block Device         0           /* DHARA FTL implementation, partition
type block device, usually a read-write FAT file system */
root     Block Device         1           /* Mini FTL implementation, partition
type block device, usually a read-only FAT file system */
spinand  MTD Device           0           /* SPI Nand MTD device, partition
read/write ultimately accesses this node to complete read/write/erase operations
*/
```

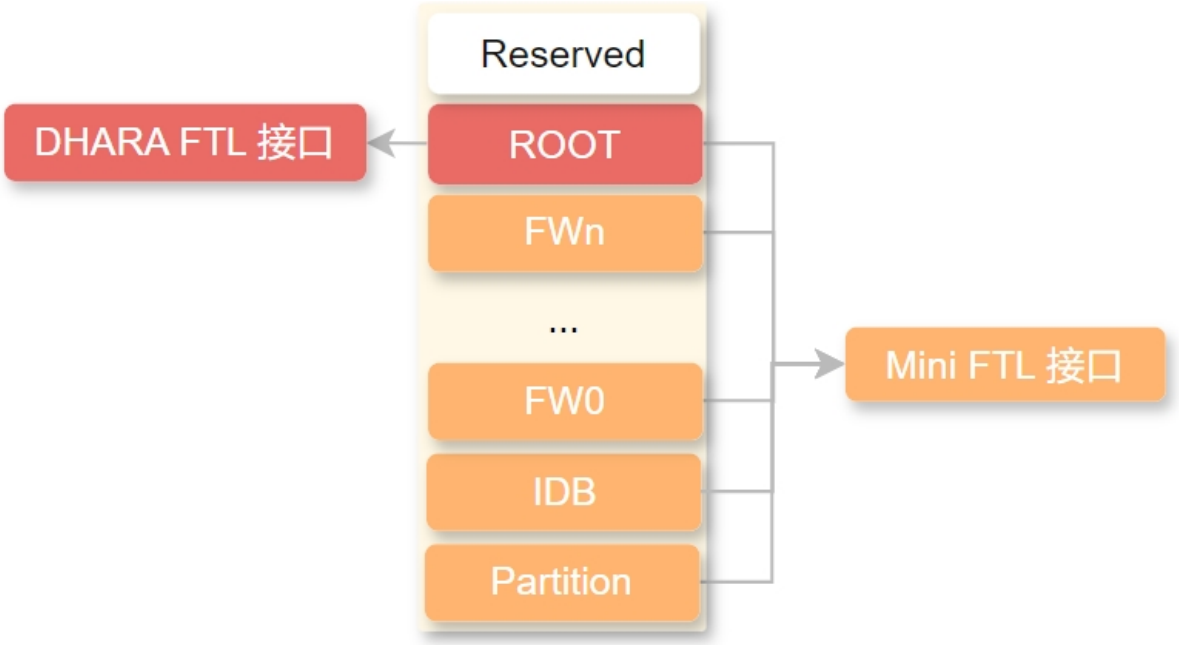
3.3 PC Tool Burning

Utilizing the SocToolKit tool, which supports both Windows and Linux versions, along with corresponding mass production tools.

Special Logic Processing:

- It is recommended to support A/B partition planning for OTA
- Regardless of whether it is a separate image or a packaged image, the final SPI Nand upgrade principle is "burning the corresponding partition image according to the partition"
    - RK Partition partition extension, currently supports separate image burning, does not support Firmware.img upgrade scheme
    - GPT Partition partition extension, supports separate image burning, update.img image burning
    - Other partition extensions, only support separate image burning

3.4 Function Interface Invocation Example



3.4.1 MTD Byte Device - Basic Interface

Refer to the spinand_test.c source code for the spinand_test test code, which mainly involves:

```
mtd_dev = (struct rt_mtd_nor_device *)rt_device_find("spinand0");
rt_mtd_nand_check_block(mtd_dev, block);
rt_mtd_nand_erase_block(mtd_dev, block);
rt_mtd_nand_write(mtd_dev, page_addr + i, (const rt_uint8_t *)buffer, mtd_dev-
>page_size, RT_NULL, 0);
rt_mtd_nand_read(mtd_dev, page_addr + i, (rt_uint8_t *)buffer, mtd_dev-
>page_size, RT_NULL, 0);
```

Explanation:

- spinand0 is a byte device
- rt_mtd_nand_check_block: Check if the block is a bad block, true for bad block, false for good block
- rt_mtd_nor_erase_block: The unit is block
- rt_mtd_nor_write/read: The unit is bytes, with the minimum alignment being pagesize, typically 2KB/4KB, for details refer to the flash manual
- RK partition table unit is sector (512B), when reading and writing partition table information, please pay attention to unit conversion.

3.4.2 Mini FTL Interface - OTA Interface/File System

OTA Interface

Refer to the test code `mini_ftl_test_one_flash_block` in the spinand_test.c source code, which mainly involves:

```
rt_uint32_t block_size = mtd_dev->pages_per_block * mtd_dev->page_size;
mtd_dev = (struct rt_mtd_nor_device *)rt_device_find("spinand0");
mini_ftl_erase(mtd_dev, addr, block_size);
mini_ftl_write(mtd_dev, buffer, addr, block_size);
mini_ftl_read(mtd_dev, buffer, addr, block_size);
```

Description:

- mini_ftl_erase: The unit is byte, and the minimum alignment is blocksize.
- mini_ftl_write: The unit is byte, and the minimum alignment is pagesize. The erase interface must be called before writing data.
- mini_ftl_read: The unit is byte, and the minimum alignment is pagesize.
- It is recommended for OTA upgrades to:
    - Align the buffer to block_size and complete erase/write/read in one call, with verification of write-read back.
    - Erase the entire partition when upgrading elm FatFs.
    - RK Partition partition table information refers to the rk_partition_list interface in the source code.
    - GPT partition table information refers to the dfs_part_list interface in the source code, name info is not supported at the moment.

File System Interface

Refer to the "partition table configuration", by default, after identifying the partition named "root" in the partition table, it registers the "root" block device and uses the Mini FTL interface to mount a read-only file system. Only supports image burning and does not support online formatting.

3.4.3 DHARA FTL Interface - File System Interface

Refer to the "Partition Table Configuration", by default, the block device with the same name as the last partition in the partition table is recognized and registered after the partition table is identified. The DHARA FTL interface is used for mounting a read-write file system. It supports image burning/online formatting.

3.5 File System

3.5.1 elm FatFs Support

RK SPI Nand Solution Provides:

- A block device based on the Mini FTL interface, read-only, supporting mounting of FAT file systems, with the firmware image being a FAT image, and does not support online low-level formatting.
- A block device based on the DHARA FTL interface, read-write capable, supporting mounting of FAT file systems, further encapsulated from the FAT image into a DHARA image firmware, with the firmware image being a DHARA image firmware, and supports online low-level formatting.

3.5.1.1 Partition Mounting

If the macro for automatic partition mounting of the file system `CONFIG_RT_USING_DFS_MNTTABLE` is enabled, you can add the corresponding partition registration information in mnt.c, for example, mounting the "root" partition to the "/" directory:

```
const struct dfs_mount_tbl mount_table[] =
{
    {"root", "/", "elm", 0, 0},
    {0}
};
```

If you wish to design your own file system mounting process, you can also achieve file system mounting through the following code:

```
dfs_mount("root", "/", "elm", 0, 0)
```

Partition mounting command:

```
mount root / elm
```

3.5.1.2 Online Low-Level Formatting of Partitions

```
mkfs -t elm userdata
```

Description:

- The root partition is a read-only partition, only supports mounting, and does not support online low-level formatting.

3.5.1.3 Offline Pre-creation of elm-fat Image

Offline Pre-creation of elm-fat Image:

It is recommended to refer to the `./bsp/rockchip/tools/mkroot.sh` script for implementation, with the key command `mkfs.fat`, where the `-S` parameter is modified to align with the page_size, typically 2048/4096, based on the SPI Nand chip manual.

3.5.1.4 Offline Pre-fabrication of DHARA Images

DHARA Tool:

```
Usage: ./dhara_utils <file_path> <flash_page_size> <num_blocks>
```

Command Example: Offline creation of FAT16 DHARA image, assuming partition size is 504 blocks, SPI Nand page size is 2KB:

```
mkfs.vfat -F 16 -s 1 -S 2048 fat.img
./rkbin/dhara_utils fat.img 2 504
```

Description:

- Output File: fat.img.dhara
- Partition size reference "Partition Planning" instructions

3.6 Test-Driven Development

3.6.1 Test Driven Infrastructure

```
    RT-Thread bsp test case  --->
        RT-Thread Common Test case  --->
            [*] Enable BSP Common TEST
            [*]   Enable BSP Common SPINAND TEST (NEW)
```

If the configuration is successful, there will be a `spinand_test` command option in msh:

```
msh />spinand_test
1. spinand_test write <page_addr> <page_num>
2. spinand_test read <page_addr> <page_nun>
3. spinand_test erase <blk_addr>
4. spinand_test erase_all <blk_addr>
5. spinand_test stress <loop>
6. spinand_test bbt
7. spinand_test markbad <blk_addr>
8. spinand_test mini_ftl_test <byte_offset>
9. spinand_test part_list
like:
        spinand_test write 1024 1
        spinand_test read 1024 1
        spinand_test erase 16
        spinand_test erase_all 512
        spinand_test stress 5000
        spinand_test bbt
        spinand_test mini_ftl_test 67108864
        spinand_test part_list
```

3.6.2 flash_stress_test Read/Write Stress Test

Test Objective To test the stability of SPI Nand under read/write stress testing conditions.

Test Equipment Prepare 10 target devices.

Test Method

Assuming the test area is: flash block offset 512 blocks, size 256 flash blocks:

1. After the device starts, execute `spinand_test stress 5000 512 256`, and ensure that the device does not power off during the test process.

2. Record the logs during the test process.
3. Make a judgment based on the test results: if the device still works normally after 24 hours, the test passes; otherwise, the test fails.

3.6.3 Power Lost Test: Unexpected Power Loss

Test Objective Nand products often encounter unexpected power loss during operation, especially for products without a battery, where the number of unexpected power losses is relatively high. Therefore, it is recommended to conduct targeted robustness testing.

Test Equipment Prepare 10 target devices.

Test Method

1. Construct a file copy scenario.
2. Set up a power-off device to cut power after 2 seconds of supply and then restore power after 3 seconds, i.e., each cycle is 5 seconds, and conduct the test for 8 hours, which is approximately 5760 unexpected power-offs, ensuring that the device can complete power-up and perform text copy actions within 5 seconds after each power-off.
3. Record logs during the test process.
4. Make judgments based on the test results; if the device still operates normally after 24 hours, the test is passed; otherwise, the test fails.

Reference Code

Refer to "SPI Nand Unexpected Power Loss Reference Code".

3.7 Frequently Asked Questions

3.7.1 Enabling SPI Nand on RK2118 Results in System Hang

Phenomenon:

```
FW data_offset:2048
FW data_size:  195584
FW load_addr:  0x1060800
copy 1090000 20a200 400
hash_256 fail! ret:-1
MemBoot fail!

SoftReset...
```

Attempting to Disable XIP Support:

```
diff --git a/bsp/rockchip/rk2118/rtconfig.py b/bsp/rockchip/rk2118/rtconfig.py
index 2b6e696e2b..32b5117513 100644
--- a/bsp/rockchip/rk2118/rtconfig.py
+++ b/bsp/rockchip/rk2118/rtconfig.py
@@ -29,8 +29,8 @@ if os.getenv('RTT_EXEC_PATH'):
 #BUILD = 'debug'
 BUILD = 'release'
```

```
-XIP = 'Y'
-#XIP = 'N'
+#XIP = 'Y'
+XIP = 'N'
 if os.getenv('RTT_BUILD_XIP'):
     XIP = os.getenv('RTT_BUILD_XIP').upper()
```

3.7.2 SPI Nand Does Not Support RK Firmware.img Packaged Image Upgrade Solution

RK Firmware.img packaged images have the following characteristics:

- Except for the last partition, which supports being configured as an empty image, the remaining partitions are filled with 0 data in the space outside of the valid image.
- With the valid image remaining unchanged, as the partition size increases, the increase in padding data leads to a significant increase in the redundancy ratio of the Firmware.img.
- During the upgrade, the Firmware.img carries redundant data for a one-time upgrade.

SPI Nand devices have the following characteristics:

- Factory devices allow for a certain proportion of factory bad blocks, so partitions require a certain amount of spare space for bad block replacement.

Therefore, due to the above characteristics, the Firmware.img packaged padding for a single firmware package conflicts with the characteristics of SPI Nand, and the upgrade method of a single firmware cannot be used. Even if the upgrade package is a single firmware, the final upgrade still requires unpacking and upgrading only the valid image according to the partition. Similarly, the programmer does not support single-image burning.

3.7.3 SPI Nand Power Loss Issues

Key Indicators of Power Loss

Unexpected power loss in devices, which refers to power down behavior without going through the standard system de-initialization process, may cause Nand chips to operate at unstable voltage levels.

The main impacts are on Nand flash erase and programming operations. Typically, erase operations take several milliseconds, while programming operations take hundreds of microseconds. 3V3 chips usually operate at voltage levels of 2.7V or higher, with the actual values based on the manual.

Typical Abnormalities

Abnormal power loss in Flash can lead to the following two situations:

- Incomplete data being written
- During power loss, if the flash is processing commands and operating at unstable voltages, it may cause random anomalies

Protection Mechanisms

Incomplete data being written:

- The file system has a power loss recovery mechanism to ensure that the file system is not affected by incomplete data, which may result in data loss

Random anomalies caused by unstable low voltage:

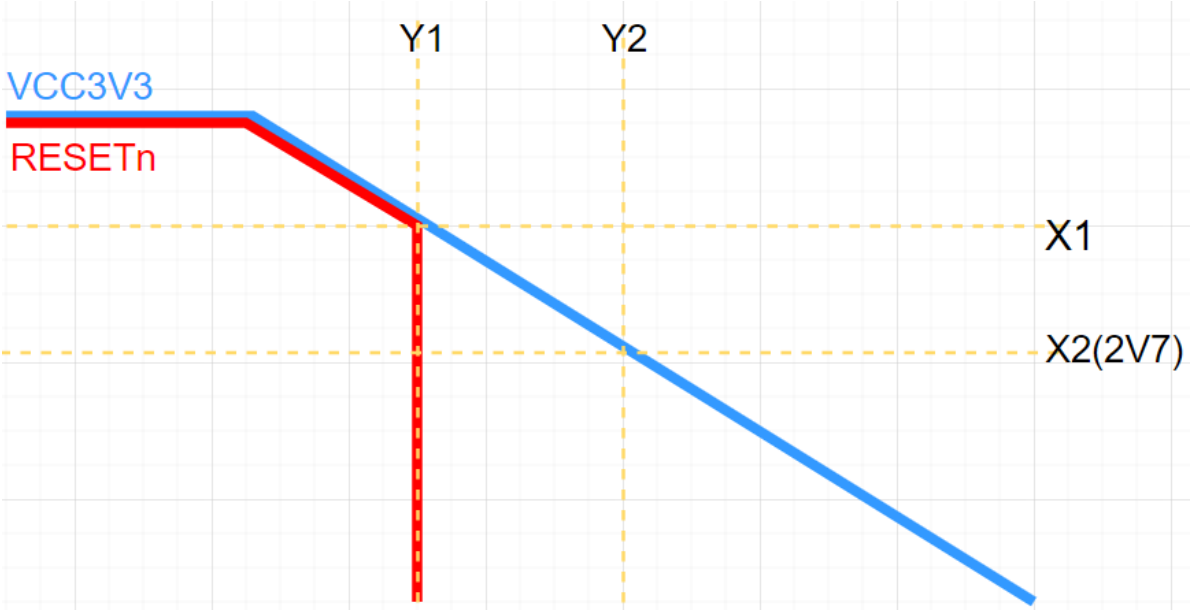
- PMIC Solution: PMIC detects VCC IO, and when the voltage level drops below a certain value, typically 2.91V, it triggers a CPU reset signal. The Host no longer initiates Nand erase/write commands, and the flash peripheral circuit enters a discharge process, maintaining a working voltage for about 1ms to ensure the last data operation of Nand is normal.
- Reset IC Solution: The principle is the same as the PMIC solution, usually choosing to detect VCCIO below the threshold voltage of 2.93V to trigger a CPU reset signal.
- RK NPOR Solution: The principle is the same as the PMIC solution, where NPOR logic detects VCCIO below the threshold voltage and triggers a CPU reset signal.

How to Avoid Abnormal Power Loss Issues

- It is recommended that Nand product designs include a battery or a constantly powered scheme to avoid abnormal power loss behavior.
- If abnormal power loss behavior cannot be avoided, it is recommended to optimize the hardware design to protect the working voltage of Nand during power down:
    - PMIC/Reset IC/RK NPOR Solution detects VCC IO voltage, monitors power loss behavior, triggers CPU reset, and avoids initiating Nand erase/write commands in low voltage environments.
    - Refer to the RK demo board and add a certain amount of capacitance to maintain the normal working voltage of Nand supply VCCIO flash, typically for a duration of tBERS, which is the block erase time.
    - Reduce power loss behavior during flash read/write processes.
    - Implement backup and recovery mechanisms for key data:
        - Firmware dual backup
        - Key data stored in read-only file systems
        - If the writable partition experiences a power loss anomaly, it is recommended to have a recovery mechanism to ensure the device can operate normally or to implement OTA.

How to Determine if the Power Loss Timing is Reasonable

In actual product and business environments, set up a "power_lost_test" testing environment, and then capture the timing of VCC3V3/RESETn signals during power loss:



VCC3V3/RESETn Signal Timing Measurement Requirements

- The oscilloscope interface provides the following information:
  - RESETn trigger point or RESET IC trigger point (Y1 axis)
  - VCC3V3 power down to 2V7 (Y2 axis)
  - Confirm the ΔY1Y2 axis time
- Measure multiple times and select the scenario with the smallest ΔY1Y2 value for analysis
- For RKNPOR solutions like RV1106, use GPIO to replace and align the RESETn signal when measuring power loss timing. The RKNPOR RESETn signal is an internal signal of the chip and cannot be directly measured. Therefore, it is recommended to consider setting a GPIO high, then adding an external pull-down, and when the CPU reset is triggered, the corresponding GPIO becomes ineffective and is pulled down by the external pull-down. The point of voltage level change can be approximated as the CPU reset trigger point.

3.7.4 SPI Nand Power Failure Reference Code

```
#include <dfs_file.h>

static void readwrite_test(void *param)
{
    int fd, size, len = 2048, loops = 0;
    char *pwrite, *pread;

    pwrite = rt_malloc(len);
    if (!pwrite) {
        rt_kprintf("malloc pwrite failed\n");
        return;
    }
    pread = rt_malloc(len);
    if (!pread) {
        rt_kprintf("malloc pread failed\n");
        rt_free(pwrite);
        return;
    }
```

```
    fd = open("/file_stress.txt", O_RDWR | O_CREAT);
    if (fd < 0)
    {
        rt_kprintf("%s open file failed\n", __func__);
        goto err_open;
    }

    while (1) {
        rt_kprintf("file_stress, loops=%d\n", loops);
        rt_memset(pwrite, loops & 0xff, len);

        lseek(fd, 0, SEEK_SET);
        write(fd, pwrite, len);
        lseek(fd, 0, SEEK_SET);
        size = read(fd, pread, len);
        if (size < 0) {
            break;
        }

        if (rt_memcmp(pwrite, pread, len)) {
            rt_kprintf("%s compare failed, %x\n", __func__, pread[0]);
            break;
        }
        loops++;
    }

    close(fd);
err_open:
    rt_free(pread);
    rt_free(pwrite);
}

void file_stress(void)
{
    rt_thread_t thread;

    thread = rt_thread_create("file_stresss",
                              readwrite_test, NULL,
                              2048,
                              21, 20);
    rt_thread_startup(thread);
}
```